

UNIVERSIDAD DEL VALLE DE GUATEMALA

Construcción de Compiladores

Sección 30



Compiscript: Analizador Léxico, Sintáctico y Semántico con
ANTLR4 y Java Swing

Jonathan Zacarías - 23401

Diego Rosales - 23258

Diego Lopez - 23242

GUATEMALA, 25 de julio 2026

Documentación Técnica y Memoria de Proyecto

Asignatura: Construcción de Compiladores

Proyecto: Laboratorio 01 & 02 — Frontend de Compilador e IDE Interactivo

Lenguaje Fuente: Compiscript (.cps)

Tecnologías: Java 21 LTS, ANTLR 4.13.2, Java Swing (Flat/Dark UI), JUnit 5, Maven

Demostración en Video: Enlace de Demostración Canva (URL omitida por requerimientos técnicos)

Fecha: Septiembre 2026

Tabla de Contenidos

1. Introducción y Objetivos
2. Especificación del Lenguaje Compiscript
3. Arquitectura Global del Compilador
4. Fase Léxica y Sintáctica (ANTLR4)
5. Fase Semántica y Sistema de Tipos
6. Catálogo Exhaustivo de Errores y Diagnósticos
7. Interfaz Gráfica de Usuario (IDE Swing)
8. Estrategia de Pruebas y Aseguramiento de Calidad
9. Guía de Compilación, Ejecución y Uso
10. Conclusiones

1. Introducción y Objetivos

El desarrollo de compiladores constituye uno de los pilares formativos en la ingeniería de software y ciencias de la computación, integrando conceptos formales de teoría de lenguajes, autómatas, gramáticas libres de contexto, grafos y análisis estático.

El presente proyecto consiste en el diseño, implementación y validación de un analizador léxico, sintáctico y semántico completo para el lenguaje de programación Compiscript (.cps), acoplado a un entorno de desarrollo integrado interactivo (IDE) en Java Swing con estética moderna (VS Code Dark).

Objetivos Principales:

- Construir la especificación léxica y sintáctica: Empleando ANTLR4 para formalizar una gramática libre de contexto robusta que soporte variables, constantes, tipado estático, colecciones, funciones, clases, herencia y control de flujo.
- Implementar un sistema de verificación semántica exhaustivo: Diseñado bajo el patrón de diseño *Visitor*, con verificación estática de tipos, resolución de ámbitos léxicos jerárquicos (*scoping*), manejo de clases, validación de herencia libre de ciclos, detección de código inalcanzable y consistencia de arreglos homogéneos.

- Proporcionar diagnósticos de error en lenguaje natural (Español): Sustituir los errores sintácticos crudos de ANTLR por mensajes descriptivos, indicando siempre línea base-1, columna base-1 y símbolo involucrado.
- Visualización didáctica del Árbol de Derivación (ParseTree): Generar un árbol gráfico interactivo con distinción cromática según la naturaleza del nodo sintáctico.
- Garantizar estabilidad: Validar el sistema a través de una suite automatizada de 99 pruebas en JUnit 5 y pruebas de regresión sobre casos reales de prueba.

2. Especificación del Lenguaje Compiscript

Compiscript es un lenguaje imperativo, estructurado, fuertemente tipado en su versión con anotaciones y orientado a objetos, diseñado específicamente con fines didácticos en compilación.

2.1 Tipos de Datos Primitivos y Estructurados

Tipo	Descripción	Ejemplos
integer	Números enteros con signo de 32/64 bits.	0, 42, -15
boolean	Valores de verdad lógicos.	true, false
string	Cadenas de caracteres delimitadas por comillas dobles.	"Hola Mundo", ""
null	Referencia nula; asignable a tipos objeto o arreglos.	null
void	Utilizado exclusivamente como tipo de retorno de funciones sin valor.	function log(): void
T[]	Arreglos homogéneos unidimensionales de tipo T.	[1, 2, 3] (integer[])
T[][]	Arreglos multidimensionales homogéneos.	[[1, 2], [3, 4]]

Tipo	Descripción	Ejemplos
Clase	Tipos estructurados creados mediante declaración class.	new Usuario("Ana")

2.2 Sintaxis y Estructuras de Control

- Declaraciones: let x: integer = 10;; const PI: integer = 3;.
- Condicionales: Sentencias if-else, operador ternario condicion ? exp1 : exp2, y switch-case con soporte para cláusula default.
- Bucles: while, do-while, for (let i = 0; i < 10; i++), y foreach (elemento in coleccion).
- Control de Salto: break, continue (restringidos a bucles y switch), y return (con o sin expresión).
- Manejo de Excepciones: Bloques try { ... } catch (e) { ... }.

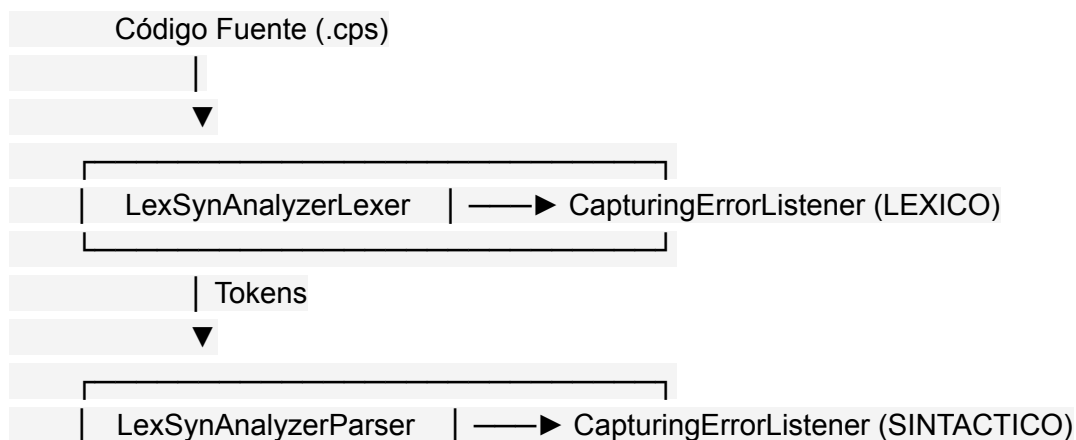
2.3 Programación Orientada a Objetos

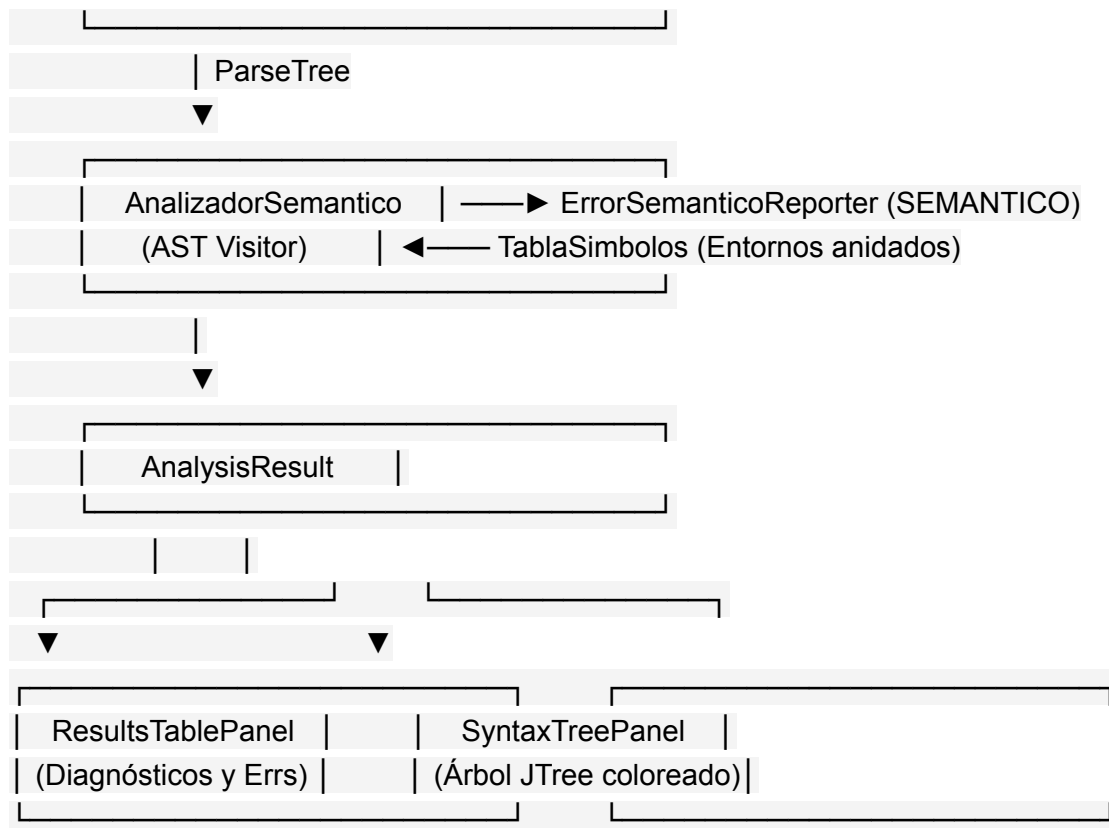
- Declaración de Clases: class NombreClase { ... }.
- Herencia Simple: class Subclase : Superclase { ... }.
- Constructores: Declarados como métodos function constructor(...) { ... } o asumidos implícitamente sin parámetros.
- Acceso a Miembros: Operador punto (instancia.atributo, instancia.metodo(...)).
- Puntero de Instancia: this para referenciar al objeto en curso (prohibido fuera de clases).

3. Arquitectura Global del Compilador

El sistema adopta una arquitectura en capas desacoplada que separa estrictamente la lógica interna del compilador de la capa visual de presentación.

3.1 Pipeline de Procesamiento





3.2 Desacoplamiento entre Motor e Interfaz de Usuario

- Independencia de Frameworks: Los paquetes `com.lexsynanalyzer.analyzer` y `com.lexsynanalyzer.semantic` no contienen importaciones de `java.awt.*` ni `javax.swing.*`. Pueden ejecutarse en entornos CLI, servidores web o pipelines de integración continua (CI/CD).
- Transporte Inmutable (`AnalysisResult`): Un record de Java 21 encapsula:
 1. `List<AnalysisError>`: Colección ordenada de todos los fallos detectados.
 2. `ParseTree`: La raíz del árbol de análisis sintáctico generado por ANTLR.
 3. `String[]`: Nombres canónicos de las reglas gramaticales para su visualización amigable.
- Ejecución Condicional: Si el análisis léxico o sintáctico reporta al menos un error crítico, el análisis semántico se omite. Esto evita la emisión de falsos positivos semánticos debidos a un árbol incompleto o malformado.

4. Fase Léxica y Sintáctica (ANTLR4)

La fase de frontend utiliza la herramienta ANTLR 4.13.2 con una gramática combinada

LexSynAnalyzer.g4.

4.1 Gramática Formal y Diseño de Tokens

- Tokens Léxicos:
 - Palabras clave: let, const, function, class, if, else, while, do, for, foreach, switch, case, default, break, continue, return, try, catch, new, this, etc.
 - Tipos base: integer, boolean, string, void.
 - Operadores: Aritméticos, Relacionales, Igualdad, Lógicos, y Asignación.
 - Delimitadores: Llaves { }, Corchetes [], Paréntesis (), Punto y coma ;, Coma ,, Dos puntos :, Punto ..
 - Literales: ENTERO, CADENA, BOOLEANO, NULL.
 - Identificadores: [a-zA-Z_][a-zA-Z0-9_]*.
 - Comentarios: De una línea // y multilínea /* ... */ redirigidos al canal -> skip.

4.2 Manejo y Normalización de Errores Sintácticos

ANTLR implementa por defecto el reporte en consola estándar con coordenadas base-0. Se implementó CapturingErrorListener extendiendo BaseErrorListener:

```
public class CapturingErrorListener extends BaseErrorListener {
    private final TipoError tipo;
    private final List<AnalysisError> errores = new ArrayList<>();

    @Override
    public void syntaxError(Recognizer<?, ?> recognizer, Object offendingSymbol,
                           int line, int charPositionInLine,
                           String msg, RecognitionException e) {
        String simbolo = offendingSymbol instanceof Token t ? t.getText() : "?";
        String descripcion = MensajesEspanol.traducir(msg, simbolo);
        errores.add(new AnalysisError(tipo, line, charPositionInLine + 1, simbolo,
        descripcion));
    }
}
```

4.3 Traducción de Diagnósticos a Español

La clase utilitaria MensajesEspanol interpreta los patrones comunes generados por ANTLR y los traduce a descripciones comprensibles para el usuario:

- *Mismatched input*: Se traduce como "Se encontró '<simbolo>', pero se esperaba...".

- *Missing token*: Se traduce como "Falta el símbolo '<esperado>' antes de...".
- *Extraneous input*: Se traduce como "Símbolo inesperado '<simbolo>'...".
- *No viable alternative*: Se traduce como "Estructura no válida cerca de '<simbolo>'...".

5. Fase Semántica y Sistema de Tipos

El análisis semántico es realizado por la clase `AnalizadorSemantico`, la cual hereda de `LexSynAnalyzerBaseVisitor<TipoDato>`.

5.1 Estructura del Sistema de Tipos (`TipoDato`)

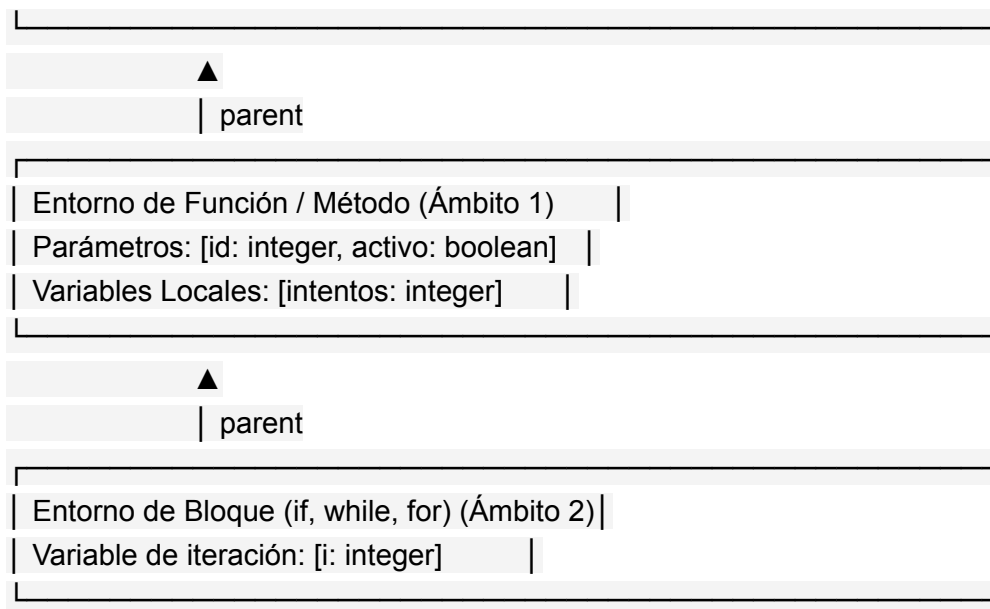
```
public class TipoDato {
    public static final TipoDato INTEGER = new TipoDato("integer");
    public static final TipoDato BOOLEAN = new TipoDato("boolean");
    public static final TipoDato STRING = new TipoDato("string");
    public static final TipoDato VOID = new TipoDato("void");
    public static final TipoDato NULL = new TipoDato("null");
    public static final TipoDato ERROR = new TipoDato("<error>");
    public static final TipoDato DESCONOCIDO = new TipoDato("<desconocido>");

    public static TipoDato arregloDe(TipoDato elemento);
    public static TipoDato deClase(String nombreClase);
    public boolean esAsignableA(TipoDato destino, TablaSimbolos tabla);
}
```

- **Compatibilidad con NULL**: El tipo NULL es asignable a cualquier instancia de clase o tipo arreglo, pero no a tipos primitivos (integer, boolean, string).
- **Subtipado por Herencia**: Si la clase Perro hereda de Animal, un objeto de tipo Perro es asignable a un destino de tipo Animal.
- **Supresión de Errores en Cascada**: Cuando una subexpresión evalúa a `TipoDato.ERROR`, los operadores padre suprimen nuevos mensajes diagnósticos para evitar la inundación del registro.

5.2 Entornos y Tabla de Símbolos (`TablaSimbolos` y `Entorno`)

Entorno Global (Ámbito 0)	
Clases: [Usuario, Factura]	
Funciones: [calcularTotal, imprimir]	
Variables Globales: [contador, modoDebug]	



- Sombreado de Identificadores (*Shadowing*): Se permite declarar una variable con el mismo nombre en un ámbito interno, ocultando la del ámbito externo sin provocar colisión.
- Detección de Redeclaraciones: En el *mismo* ámbito léxico, una redeclaración provoca error semántico inmediato.

5.3 Fase de Pre-declaración y Resolución Adelantada

Antes de validar el cuerpo de un bloque o programa, el visitor realiza un barrido previo registrando la signatura de todas las funciones y clases. Esto permite:

1. Llamadas a funciones declaradas posteriormente en el archivo.
2. Recursión simple y recursión mutua.
3. Instanciación y herencia de clases declaradas en líneas posteriores.

5.4 Validación de Expresiones y Operadores

1. Aritméticos (+, -, *, /, %): Requieren operandos de tipo integer. Caso especial: + admite concatenación si ambos operandos son string.
2. Lógicos (&&, ||, !): Requieren estrictamente operandos boolean.
3. Relacionales (<, <=, >, >=): Requieren operandos integer y retornan boolean.
4. Igualdad (==, !=): Requieren operandos de tipos compatibles y retornan boolean.
5. Operador Ternario (cond ? a : b): La condición debe ser boolean. Ramas verdadera y falsa deben ser compatibles.

5.5 Validación de Clases, Herencia y Detección de Ciclos

- Herencia Válida: Verifica que la clase padre exista y sea categoría CLASE.
- Detección de Herencia Cíclica: Previene heredar de sí misma directa o indirectamente.
- Acceso a Miembros: Se busca en la tabla de miembros de la clase y antecesoras.
- Uso de this: Solo admitido dentro de métodos o constructores.

5.6 Colecciones, Arreglos Homogéneos y Bucle foreach

- Homogeneidad Estricta: Los elementos deben ser del mismo tipo.
- Arreglos Multidimensionales: Se validan como `integer[][]`, rechazando mezclas.
- Indexación (`arr[i]`): Receptor debe ser arreglo o cadena; índice debe ser integer.
- Sentencia foreach: Colección debe ser arreglo o cadena; declara variable de iteración local automáticamente.

5.7 Control de Flujo y Análisis de Código Inalcanzable

- Restricción de return: Solo dentro de funciones o métodos. Retorno obligatorio si tipo no es void.
- Restricción de break y continue: Solo permitidos en bucles o switch.
- Código Inalcanzable (*Dead Code*): Reporta advertencia/error tras saltos incondicionales.

6. Catálogo Exhaustivo de Errores y Diagnósticos

Categoría	Situación Anómala	Mensaje Diagnóstico Emitido
Léxico	Carácter desconocido	Carácter no reconocido: '<carácter>'
Léxico	Cadena sin cerrar	Cadena de texto sin cerrar al final de la línea
Sintáctico	Estructura malformada	Se encontró '<símbolo>', pero se esperaba '<token>'
Semántica - POO	Herencia cíclica	Herencia cíclica detectada...
Semántica - Tipos	Asignación incompatible	La variable '<v>' es de tipo '<T1>' y no puede recibir un

Categoría	Situación Anómala	Mensaje Diagnóstico Emitido
		valor de tipo '<T2>'.
Semántica - Flujo	Código inalcanzable	Esta sentencia nunca se ejecuta: el flujo ya termina antes de llegar aquí.

7. Interfaz Gráfica de Usuario (IDE Swing)

La interfaz fue diseñada con una estética profesional inspirada en el tema oscuro de editores modernos como Visual Studio Code.

7.1 Editor de Código Fuente con Numeración y Resaltado

- Área de Texto (JTextArea): Tipografía monoespaciada (Consolas / JetBrains Mono).
- Gutter con Numeración: Enumera dinámicamente cada línea del archivo fuente.
- Sincronización Bidireccional de Errores: Al seleccionar un error, el cursor se desplaza a la línea y la resalta visualmente.

7.2 Visualizador Interactivo del ParseTree (SyntaxTreePanel)

- Diferenciación Cromática de Nodos: Distintos colores para reglas sintácticas, palabras clave, literales, identificadores y operadores.
- Herramientas de Exploración: Botones Expandir Todo y Colapsar Todo, con contador de nodos en tiempo real.

7.3 Navegación Asíncrona con SwingWorker

El pipeline de compilación se delega a un hilo de trabajo (SwingWorker) para prevenir el bloqueo de la interfaz durante el análisis de archivos extensos.

8. Estrategia de Pruebas y Aseguramiento de Calidad

El proyecto implementa una rigurosa suite de pruebas automatizadas utilizando JUnit 5.

8.1 Cobertura de Pruebas Unitarias JUnit 5

La suite abarca 99 pruebas automatizadas distribuidas en paquetes temáticos:

CasosPruebaTest, ClasesSemanticasTest, ArreglosSemanticosTest, TiposSemanticosTest, FuncionesSemanticasTest, AmbitosSemanticosTest, FlujoSemanticoTest, SyntaxTreePanelTest.

8.2 Batería de Regresión con los 8 Casos Oficiales

Archivo de Prueba	Complejidad	Errores Esperados	Estado del Compilador
baja_sin_errores.cps	Baja	0 errores	Pasó exitosamente
baja_errores_lexicos.cps	Baja	Errores de caracteres inválidos	Identificados (LÉXICO)
baja_errores_mixto.cps	Baja	Errores léxicos y sintácticos	Capturados en orden
media_sin_errores.cps	Media	0 errores	Pasó exitosamente

9. Guía de Compilación, Ejecución y Uso

9.1 Requisitos de Entorno

- JDK 21 o superior.
- Apache Maven 3.8+.

9.2 Comandos de Terminal

```
mvn clean compile
mvn test
mvn exec:java
```

9.3 Manual de Usuario del IDE

1. Cargar Código: Presione el botón Abrir Archivo (.cps) o utilice Ctrl+O.
2. Edición: Modifique código y guarde con Ctrl+S.
3. Compilación: Presione Analizar o F5.
4. Inspección de Diagnósticos: Use la pestaña Tabla de Errores para saltar a líneas infractoras.
5. Inspección del Árbol: Navegue la jerarquía sintáctica generada en la pestaña Árbol Sintáctico (ParseTree).

10. Conclusiones

1. Robustez y Tolerancia: Captura de todos los errores en una sola pasada.
2. Claridad Diagnóstica: Traducción contextual a español.
3. Escalabilidad del Sistema de Tipos: Arquitectura Visitor altamente modular y extensible para características POO complejas.
4. Experiencia de Usuario Integral: IDE con herramientas visuales sincronizadas aportando valor académico y profesional.